Single Cycle Check

AlgoExpert · Medium · Graphs

The problem

- Each element is a jump (wrapping).
- Do the jumps form one cycle visiting every index exactly once and returning to the start?

Intuition

- Follow the jumps n times: we must visit every index exactly once and return to the start.

Brute force vs optimal

- Visited array → O(n) space
- Count jumps instead: n jumps, no early return to 0 → O(1) space
- Floor-mod handles negative jumps

Approach

- 1. Jump n times with wrap-around
- 2. If we return to zero early, fail
- 3. After n jumps we must be at zero

Worked example




```
jump +2: 0 → 2
jumps so far: 1
```

0 → 2

Worked example




```
jump +1: 2 → 3
jumps so far: 2
```

2 → 3

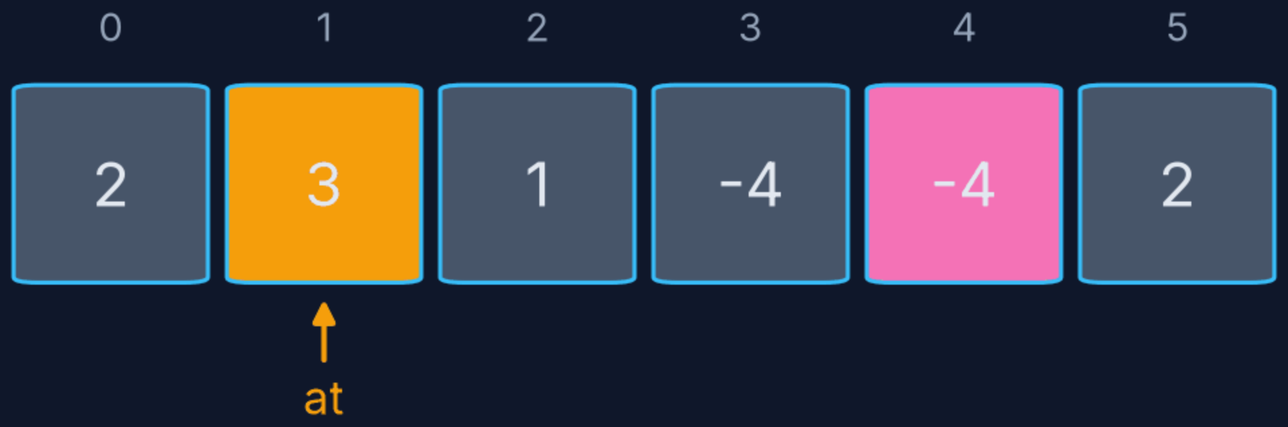
Worked example




```
jump -4: 3 → 5
jumps so far: 3
```

3 → 5

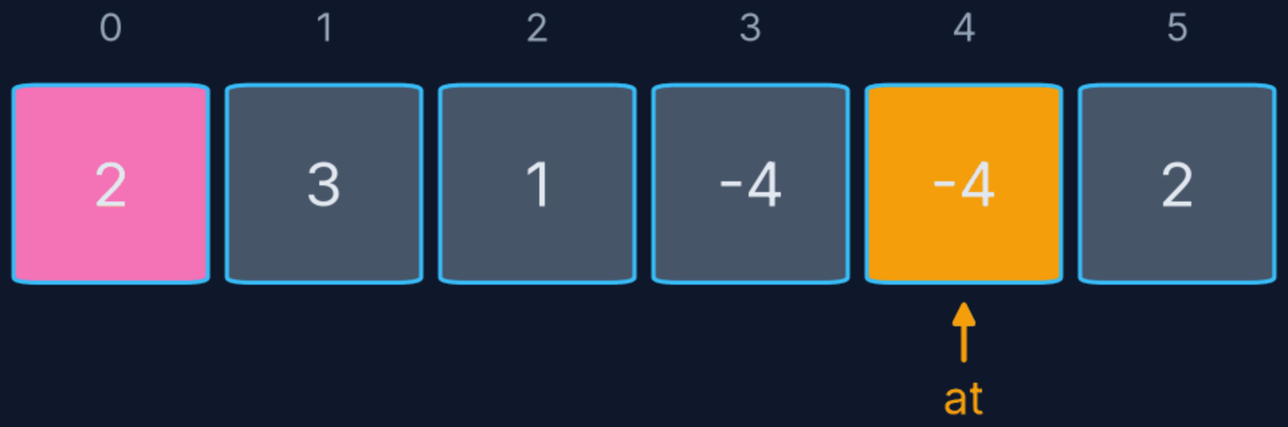
Worked example




```
jump +3: 1 → 4
jumps so far: 5
```

1 → 4

Worked example




```
jump -4: 4 → 0
jumps so far: 6
```

4 → 0

Worked example



After 6 jumps we're back at 0 → single cycle

Java solution

```java
public class SingleCycleCheck {
    public static boolean hasSingleCycle(int[] a) {
        int n = a.length, idx = 0;
        for (int visited = 0; visited < n; visited++) {
            if (visited > 0 && idx == 0) return false;
            idx = Math.floorMod(idx + a[idx], n);
        }
        return idx == 0;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Negative jumps: use floorMod
- Large jumps wrap multiple times
- Returning to 0 early → false

Interview tips & follow-ups

- Explain why n jumps + end at 0 suffices
- Follow-up: detect any cycle from any start

Recap

- Follow the jumps n times: we must visit every index exactly once and return to the start.
- Complexity: O(n)